

Explicación del Código: mano_robotica_wifi_pura_matematica.py

Este documento explica de manera detallada cómo funciona el script principal que utilizas para controlar la mano robótica. El código utiliza **Visión Artificial (MediaPipe)** para detectar tu mano a través de la cámara del ESP32, calcula matemáticamente qué dedos están abiertos o cerrados, y envía esa información por **WiFi** al controlador de los servomotores.

1. Configuración WiFi

```
ESP32_IP = "192.168.4.1"
URL_VIDEO = f"http://{ESP32_IP}:81/stream"
URL_COMMAND = f"http://{ESP32_IP}:82/dedos?estado="
```

Aquí se definen las direcciones para comunicarse con el ESP32-CAM:

- **URL_VIDEO:** Es la dirección de donde Python "descarga" el video en vivo de la cámara.
- **URL_COMMAND:** Es la dirección a la que Python envía las instrucciones (ej. 1, 0, 0, 0, 0 para cerrar 4 dedos).
- La función `send_wifi_command(state)` se encarga de enviar la petición de forma rápida (con un tiempo límite de 0.1 segundos) para que el programa no se quede congelado si la red está lenta.

2. Modelo de MediaPipe (Inteligencia Artificial)

```
options = HandLandmarkerOptions(...)
detector = HandLandmarker.create_from_options(options)
```

El código utiliza el modelo **Hand Landmarker de MediaPipe**, que es capaz de detectar 21 puntos clave (landmarks) en tu mano.

- Se configura en modo `LIVE_STREAM` (transmisión en vivo), lo que significa que procesa los fotogramas del video a medida que llegan.
- La función `on_result` se llama automáticamente cada vez que MediaPipe termina de analizar una imagen, guardando el resultado en la variable `latest_result` para que el resto del programa la use.

3. Funciones de Dibujo

```
def draw_hand(frame, landmarks, w, h):
```

Esta función toma los 21 puntos detectados por MediaPipe y dibuja líneas verdes conectando las articulaciones (creando un "esqueleto" de tu mano en la pantalla) y pequeños círculos en las puntas de los dedos para que puedas ver exactamente lo que la cámara está detectando.

4. Conexión a la Cámara y Ventana

```
cap = cv2.VideoCapture(URL_VIDEO)
cv2.namedWindow("WiFi Hand", cv2.WINDOW_NORMAL)
cv2.resizeWindow("WiFi Hand", 1280, 960)
```

Utilizando la librería OpenCV (`cv2`), el programa se conecta al flujo de video del ESP32. Además, crea la ventana donde verás tu mano y ajusta su tamaño a 1280x960 píxeles, para que la imagen se vea lo suficientemente grande en tu pantalla.

5. El Bucle Principal (La Lógica Matemática)

El programa entra en un bucle `while True` (se repite infinitamente hasta que lo cierras) donde hace lo siguiente por cada fotograma que llega de la cámara:

A. Preparación de la imagen

La imagen se voltea (efecto espejo con `cv2.flip`) para que sea más intuitiva de ver, y se envía al detector de MediaPipe (`detector.detect_async`).

B. Sistema de Pausa

Si presionas la barra espaciadora (`is_paused`), el sistema muestra en pantalla "=== SYSTEM PAUSED ===" y fuerza el estado a "1,1,1,1,1" (mano totalmente abierta), deteniendo temporalmente el control de la mano robótica.

C. La "Pura Matemática" (Detección de dedos)

Si detecta una mano, el código no usa una red neuronal compleja para saber si el dedo está abierto, sino que usa **distancias matemáticas**:

1. **El Pulgar:** Mide la distancia desde la punta del pulgar hasta la base del dedo meñique.
2. **Los otros 4 dedos:** Miden la distancia desde la punta de cada dedo hasta la muñeca.

El filtro "Histéresis" (Evitar temblores):

Para evitar que los dedos de la mano robótica tiemblen o parpadeen rápidamente cuando tienes la mano a medio cerrar, el código tiene "memoria" (`previous_fingers_state`):

- Si el dedo está **abierto**, requiere que lo cierres bastante (pasar de un umbral negativo) para considerar que lo cerraste.
- Si el dedo está **cerrado**, requiere que lo abras bastante (pasar de un umbral positivo) para considerar que lo abriste. Esto crea un movimiento mucho más estable y sólido.

D. Envío de Comandos en Segundo Plano (Threading)

```
if state != last_sent_state and (current_time - last_send_time) > send_interval:
    threading.Thread(target=send_wifi_command, args=(state,), daemon=True).start()
```

Para que el video no se trabe mientras se envía la orden por WiFi al ESP32, el código crea un "hilo" (Thread). Esto es como asignar un trabajador secundario para que envíe el mensaje, permitiendo que el trabajador principal siga procesando el video de la cámara sin pausas. Solo envía un comando si el estado de la mano cambió y si ha pasado al menos 0.1 segundos desde el último envío.

6. Salida Segura (Exit Routine)

```
urllib.request.urlopen(URL_COMMAND + "1,1,1,1", timeout=1.0)
```

Si presionas la tecla `q`, el bucle se rompe. Antes de cerrar el programa por completo, el código hace un último esfuerzo enviando un comando forzado para abrir toda la mano (1,1,1,1). Esto es una medida de seguridad para evitar que la mano robótica se quede atascada apretando algo cuando apagas el programa en tu computadora. Finalmente, libera la cámara y destruye las ventanas de OpenCV.